Práctica N°2: Control de Motor DC con incremento de velocidad en PIC32 MAX32

Índice

1. Objetivos **2**
- 1.1. Objetivo general 2
- 1.2. Objetivos específicos 2

2. Descripción del código **2**
- 2.1. Explicación paso a paso 3

3. Resultados **4**

1. Objetivos

1.1. Objetivo general

Implementar y analizar el control de velocidad de un motor de corriente continua (DC) utilizando la tarjeta **ChipKIT Max32** junto con el **Motor Shield**, aplicando la técnica de modulación por ancho de pulso (PWM) y un pulsador como interfaz de entrada para modificar dinámicamente el ciclo de trabajo.

1.2. Objetivos específicos

- Comprender el principio de funcionamiento del **PWM** y su influencia en la velocidad de un motor DC.
- Configurar correctamente los pines de salida digital y PWM en la plataforma **ChipKIT Max32**.
- Implementar un programa que incremente el valor del ciclo de trabajo del PWM mediante un **pulsador** conectado a una entrada digital.
- Analizar el comportamiento del motor frente a los cambios en el ciclo de trabajo y observar la variación de la velocidad.

2. Descripción del código

A continuación se presenta el código fuente implementado para controlar un motor de corriente continua mediante la tarjeta **ChipKIT Max32** y el **Motor Shield**. El programa permite variar el ciclo de trabajo del PWM utilizando un único pulsador conectado a una entrada digital configurada con resistencia *pull-up* interna.

```
1 // Prueba simple para motor con ChipKIT Max32 + Motor
     Shield
2 // PWM -> pin 3 (RD0 / OC1)
3 // DIR -> pin 4 (RF1)
4 // BTN -> pin 7 (a GND, use INPUT_PULLUP)  <--  nico
     bot n
5
6 const int PWM_PIN    = 3;    // RD0 / OC1
7 const int DIR_PIN    = 4;    // RF1
8 const int BTN        = 78;   // bot n (a GND)
9
```

```
10 int duty = 0;                // 0..255 (valor inicial)
11 const int STEP = 25;         // incremento por pulsaci n
12 const int DUTY_MIN = 0;
13 const int DUTY_MAX = 255;
14
15 int lastButtonState = HIGH; // estado anterior del
       bot n (INPUT_PULLUP: idle = HIGH)
16
17 void setup() {
18   pinMode(PWM_PIN, OUTPUT);
19   pinMode(DIR_PIN, OUTPUT);
20   pinMode(BTN, INPUT_PULLUP);
21
22   // Estado inicial
23   digitalWrite(DIR_PIN, LOW);
24   analogWrite(PWM_PIN, duty);
25 }
26
27 void loop() {
28   int current = digitalRead(BTN);
29
30   // Detecci n de flanco: HIGH -> LOW significa "
       pulsado"
31   if (lastButtonState == HIGH && current == LOW) {
32     duty += STEP;
33     if (duty > DUTY_MAX) duty = DUTY_MIN; // si supera
         el m ximo, reinicia
34     analogWrite(PWM_PIN, duty);
35   }
36
37   lastButtonState = current;
38   delay(100); // peque o retardo para aliviar CPU (no
       es debounce)
39 }
```

Listing 1: Código fuente para el control de un motor DC con el ChipKIT Max32 y Motor Shield

2.1. Explicación paso a paso

1. **Definición de pines:** Se definen tres pines principales:
   - `PWM_PIN = 3`: salida de modulación por ancho de pulso (PWM)

correspondiente al pin RD0/OC1 del PIC32.

- `DIR_PIN = 4`: controla la dirección del giro del motor mediante el pin RF1.
- `BTN = 78`: entrada digital conectada a un pulsador. Se utiliza la resistencia interna `INPUT_PULLUP`, por lo que el estado lógico será `HIGH` en reposo y `LOW` al presionar.

2. **Variables de control:** Se define `duty` como la variable que determina el ciclo de trabajo del PWM (rango de 0 a 255). La constante `STEP = 25` indica el incremento de velocidad en cada pulsación del botón. Cuando el valor supera `255`, se reinicia a cero.

3. **Configuración inicial (`setup()`)**: Los pines se configuran como salidas o entradas según su función. Luego se establece un estado inicial: dirección baja (`LOW`) y ciclo de trabajo inicial de 0, manteniendo el motor detenido al inicio.

4. **Bucle principal (`loop()`)**: En cada iteración se lee el estado actual del botón (`digitalRead(BTN)`). Se compara el estado anterior (`lastButtonState`) con el actual para detectar un **flanco descendente** (transición de `HIGH` a `LOW`), que representa una pulsación.

5. **Ajuste del PWM:** Cada vez que se detecta una pulsación, se incrementa el ciclo de trabajo en `STEP` unidades. Si el valor supera el máximo (`255`), se reinicia a cero, permitiendo un ciclo continuo de aumento.

6. **Actualización del motor:** El nuevo valor de `duty` se aplica con `analogWrite(PWM_PIN, duty)`, modificando la velocidad del motor.

7. **Pequeño retardo:** Se agrega un retardo de 100 ms mediante `delay(100)` para evitar lecturas excesivas y reducir la carga del procesador. Este retardo no cumple la función de *debounce*, pero mejora la estabilidad básica del programa.

3. Resultados

En la Figura 1 se muestra el resultado del funcionamiento del sistema implementado con la tarjeta **ChipKIT Max32** y el **Motor Shield**. Se observa el motor de corriente continua en operación, cuya velocidad varía conforme se presiona el pulsador, modificando el ciclo de trabajo del PWM generado por el microcontrolador.

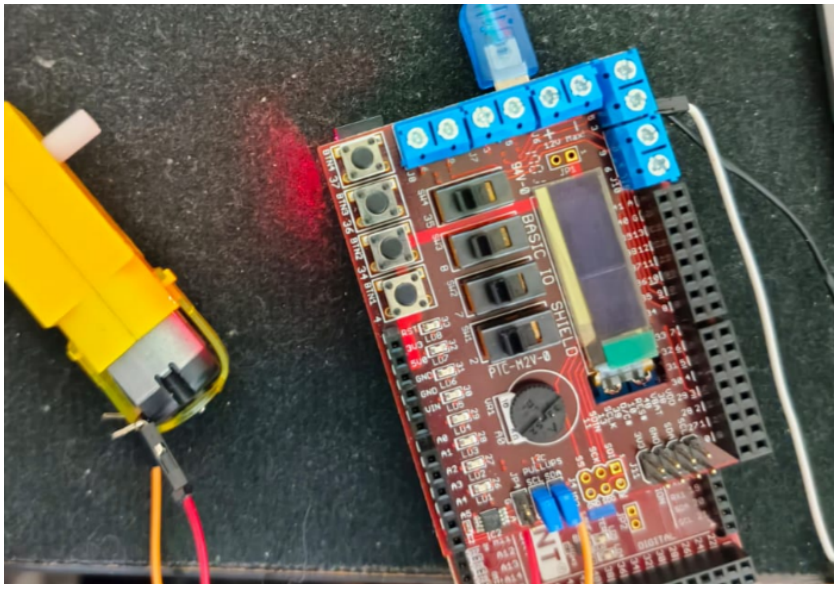


Figura 1: Funcionamiento del motor DC controlado por PWM usando la tarjeta ChipKIT Max32 y el Motor Shield.